

Enterprise Single Sign-On & Dual Authentication Integration

End-to-End Technical Implementation Guide

System	Intervention Management Platform
Architecture	ASP.NET Core 8 (MVC UI & Web API)
Identity Providers	Azure Entra ID (OIDC) & Local SQL DB
Infrastructure	Azure App Services, Key Vault, Graph API
Methodology	Clean Architecture, Zero-Downtime Rollout

1. Architectural Overview

This document serves as the master blueprint for implementing a highly secure, dual-authentication system across the Intervention Management .NET 8 UI and API applications. It incorporates native Azure Entra ID integration alongside legacy database authentication, controlled seamlessly via centralized feature flags.

1.1 Core Components

- **UI App (MVC):** Manages user sessions, dual-login routing, and securely attaches API tokens using custom DelegatingHandler middleware.
- **API App (Web API):** Validates Entra ID Bearer tokens and translates Entra Object IDs (OID) into database-driven RBAC roles via ICclaimsTransformation.
- **Key Vault:** Injected at startup via Azure Managed Identities to eliminate physical secrets.

2. Configuration & Secrets Management

Both applications utilize Azure Key Vault for secrets management and the Options pattern for feature flags.

2.1 Shared appsettings.json

```
{
  "KeyVault": {
    "VaultUri": "https://kv-intervention-dev.vault.azure.net/"
  },
  "AuthenticationFeatures": {
    "EnableSSO": true,
    "EnableLocalLogin": true
  },
  "AzureAd": {
    "Instance": "https://login.microsoftonline.com/",
    "Domain": "yourdomain.onmicrosoft.com",
    "TenantId": "YOUR_TENANT_ID",
    "ClientId": "YOUR_APP_CLIENT_ID",
    "CallbackPath": "/signin-oidc"
    // ClientSecret is mapped automatically from Key Vault
  },
  "DownstreamApi": {
    "BaseUrl": "https://api-intervention-dev.azurewebsites.net/",
    "Scopes": [ "api://intervention-api-client-id/access_as_user" ]
  }
}
```

2.2 Feature Flag Options Model

```
namespace InterventionManagement.Ui.Web.Models;

public sealed class AuthenticationFeaturesOptions
{
    public const string SectionName = "AuthenticationFeatures";
    public bool EnableSSO { get; init; } = true;
    public bool EnableLocalLogin { get; init; } = false;
}
```

3. UI Application Implementation

| 3.1 Bootstrap & Middleware (Program.cs)

The UI startup pipeline configures Key Vault, binds options, establishes the dual-authentication schema, and handles Azure App Service TLS termination requirements.

```

var builder = WebApplication.CreateBuilder(args);

// 1. Key Vault Integration (Managed Identity)
var vaultUri = builder.Configuration["KeyVault:VaultUri"];
if (!string.IsNullOrEmpty(vaultUri))
{
    builder.Configuration.AddAzureKeyVault(
        new Uri(vaultUri),
        new DefaultAzureCredential());
}

// 2. Options Binding
builder.Services.Configure<AuthenticationFeaturesOptions>(
    builder.Configuration.GetSection(AuthenticationFeaturesOptions.SectionName));

var authFeatures = builder.Configuration.GetSection(AuthenticationFeaturesOptions.SectionName)
    .Get<AuthenticationFeaturesOptions>() ?? new AuthenticationFeaturesOptions();

// Bulletproof Scope Parsing
var scopesSection = builder.Configuration.GetSection("DownstreamApi:Scopes");
var apiScopes = scopesSection.Get<string[]>()
    ?? new[] { scopesSection.Value ?? string.Empty }.Where(s => !string.IsNullOrEmpty(s)).ToArray();

// 3. Dual Authentication Setup
var authBuilder = builder.Services.AddAuthentication(options =>
{
    options.DefaultScheme = CookieAuthenticationDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = CookieAuthenticationDefaults.AuthenticationScheme;
})
.AddCookie(CookieAuthenticationDefaults.AuthenticationScheme, options =>
{
    options.LoginPath = "/Account/Login";
    options.AccessDeniedPath = "/Account/Login";
});

if (authFeatures.EnableSSO)
{
    authBuilder.AddMicrosoftIdentityWebApp(builder.Configuration.GetSection("AzureAd"))
        .EnableTokenAcquisitionToCallDownstreamApi(apiScopes)
        .AddMicrosoftGraph(builder.Configuration.GetSection("Graph"))
        .AddDistributedTokenCaches(); // Required for multi-instance App Services
}

// 4. Infrastructure & Pipeline
builder.Services.AddAuthorization();
builder.Services.AddInfrastructure(builder.Configuration, authFeatures.EnableSSO);
builder.Services.AddControllersWithViews().AddMicrosoftIdentityUI();

var app = builder.Build();

// Critical for App Service Linux containers / load balancers
app.UseForwardedHeaders(new ForwardedHeadersOptions {
    ForwardedHeaders = ForwardedHeaders.XForwardedProto | ForwardedHeaders.XForwardedHost
});

// ... Standard routing and auth middleware ...
app.UseAuthentication();
app.UseAuthorization();

```

```
app.MapControllerRoute(name: "default", pattern: "{controller=Dashboard}/{action=Index}/{id?}");
app.Run();
```

3.2 API Client Infrastructure (Dependency Injection & Middleware)

To ensure Clean Architecture, the API Client is registered as a strict Typed Client. Authentication injection is offloaded entirely to a DelegatingHandler to maintain the Single Responsibility Principle.

DependencyInjection.cs

```
public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(this IServiceCollection services, IConfiguration configuration)
    {
        services.AddHttpContextAccessor();
        services.AddTransient<TokenAuthorizationHandler>();

        // Unified Typed Client Registration
        services.AddHttpClient<IInterventionApiClient, SecureInterventionApiClient>(client =>
        {
            client.BaseAddress = new Uri(configuration["DownstreamApi:BaseUrl"]);
            ?? throw new InvalidOperationException("BaseUrl is missing.");
        })
        .AddHttpMessageHandler<TokenAuthorizationHandler>();

        return services;
    }
}
```

TokenAuthorizationHandler.cs

Dynamically resolves the correct access token based on the user's login origin.

```

public class TokenAuthorizationHandler(
    IHttpContextAccessor httpContextAccessor,
    IConfiguration configuration) : DelegatingHandler
{
    protected override async Task<HttpResponseMessage> SendAsync(HttpRequestMessage request, CancellationToken ct)
    {
        var context = httpContextAccessor.HttpContext;
        if (context?.User.Identity?.IsAuthenticated == true)
        {
            string? token = null;
            var isLocalLogin = context.User.FindFirst("idp")?.Value == "local";

            if (isLocalLogin)
            {
                // Fetch token from encrypted cookie properties
                token = await context.GetTokenAsync("access_token");
            }
            else
            {
                // Fetch from MSAL distributed cache
                var tokenAcquisition = context.RequestServices.GetService<ITokenAcquisition>();
                if (tokenAcquisition != null)
                {
                    var scopes = configuration.GetSection("DownstreamApi:Scopes").Get<string[]>() ?? [];
                    token = await tokenAcquisition.GetAccessTokenForUserAsync(scopes);
                }
            }

            if (!string.IsNullOrEmpty(token))
            {
                request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", token);
            }
        }
        return await base.SendAsync(request, ct);
    }
}

```

SecureInterventionApiClient.cs

A clean, focused HTTP client devoid of authentication logic.

```

public sealed class SecureInterventionApiClient(HttpClient client) : IInterventionApiClient
{
    public Task<IReadOnlyList<InterventionDto>> GetDashboardAsync(CancellationToken ct) =>
        GetAsync<IReadOnlyList<InterventionDto>>("api/reports/dashboard", ct);

    private async Task<T> GetAsync<T>(string path, CancellationToken ct)
    {
        return await client.GetFromJsonAsync<T>(path, ct)
            ?? throw new InvalidOperationException($"API returned no payload for {path}.");
    }
}

```

3.3 Authentication Controller (AccountController.cs)

Manages routing, secure token storage, and Identity Provider-aware sign-out.

```
[HttpPost]
public async Task<IActionResult> LocalLogin(string username, string password, string? returnUrl)
{
    var apiToken = await GetLocalApiTokenAsync(username, password); // Internal auth call

    var claims = new[] {
        new Claim(ClaimTypes.Name, username),
        new Claim("oid", "user-db-id"),
        new Claim("idp", "local") // Marker for logout routing
    };

    // Secure Token Storage to prevent Cookie Header Bloat
    var authProperties = new AuthenticationProperties();
    authProperties.StoreTokens(new[] {
        new AuthenticationToken { Name = "access_token", Value = apiToken.AccessToken }
    });

    await HttpContext.SignInAsync(
        CookieAuthenticationDefaults.AuthenticationScheme,
        new ClaimsPrincipal(new ClaimsIdentity(claims, "Cookies")),
        authProperties);

    return LocalRedirect(returnUrl ?? "/");
}

[HttpPost]
public IActionResult Logout()
{
    var isLocalLogin = User.FindFirstValue("idp") == "local";

    if (_authFeatures.EnableSSO && !isLocalLogin)
    {
        return SignOut(new AuthenticationProperties { RedirectUri = "/Account/Login" },
            CookieAuthenticationDefaults.AuthenticationScheme,
            OpenIdConnectDefaults.AuthenticationScheme);
    }

    return SignOut(new AuthenticationProperties { RedirectUri = "/Account/Login" },
        CookieAuthenticationDefaults.AuthenticationScheme);
}
```

4. API Application Implementation

The API secures endpoints via JWT validation while transparently mapping Entra ID objects to existing Database RBAC Roles.

4.1 Database Role Interception (IClaimsTransformation)

```
public class DatabaseRoleClaimsTransformation : IClaimsTransformation
{
    private readonly IUserRepository _userRepo;

    public DatabaseRoleClaimsTransformation(IUserRepository userRepo) { _userRepo = userRepo; }

    public async Task<ClaimsPrincipal> TransformAsync(ClaimsPrincipal principal)
    {
        if (principal.HasClaim(c => c.Type == "RolesMapped")) return principal;

        var identity = new ClaimsIdentity(principal.Identity);
        var oidClaim = principal.FindFirst("http://schemas.microsoft.com/identity/claims/objectidentifier")
            ?? principal.FindFirst("oid");

        if (oidClaim != null)
        {
            // Maps Entra ID to legacy DB roles e.g., "Approver", "Requester"
            var dbRoles = await _userRepo.GetUserRolesByProviderIdAsync(oidClaim.Value);
            foreach (var role in dbRoles)
                identity.AddClaim(new Claim(ClaimTypes.Role, role));
        }

        identity.AddClaim(new Claim("RolesMapped", "true"));
        return new ClaimsPrincipal(identity);
    }
}
```

5. Team Rollout & App Service Deployment Strategy

To support an active team of 4 developers across 5 environments without disrupting ongoing work, a Zero-Impact Slot Deployment strategy is utilized.

The "Dark Launch" Strategy

The code is merged to main with EnableSSO = false by default. Developers continue using standard DB auth locally. Environments are controlled dynamically via Azure App Service Configuration "Sticky" Slot settings.

Environment	Azure Slot	EnableSSO	Purpose
Local Dev	N/A	False	Team continues building features uninterrupted via Local DB Auth.
Dev	dev-slot	True	Integration testing of Entra ID Flow.
Staging		True	QA Validation.

Environment	Azure Slot	EnableSSO	Purpose
	staging-slot		
UAT / PreProd	uat-slot	False	Awaiting client sign-off. Flipped to True dynamically when ready.
Production	prod-slot	False	Zero-Downtime Go-Live. Switched to True via Azure Portal settings on launch day.